SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)**
Weightage:	**30%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	YAZHINI.E.M	**Reg.No.:**	192511165
Department:	B.E.CSE	**Date:**	29/08/2026

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I YAZHINI.E.M /192511165 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Question 1 — DMA-Based Data Transfer Algorithm

Task: Design pseudocode that moves data from an I/O device to memory using DMA, handles interrupts, and minimizes CPU involvement.

Pseudocode:

```
BEGIN DMA_TRANSFER(device, memory_addr, length)
    // Step 1: CPU configures DMA controller (one-time setup)
    DMA.source        <- device.data_register
    DMA.destination   <- memory_addr
    DMA.transfer_len  <- length
    DMA.mode          <- BLOCK_TRANSFER
    ENABLE_INTERRUPT(DMA.completion_line)

    // Step 2: CPU hands off control to DMA and continues other work
    DMA.START()
    CPU.resume_other_tasks()      // CPU is NOT blocked

    // Step 3: DMA engine autonomously moves data
    WHILE DMA.bytes_remaining > 0 DO
        DMA.transfer_next_chunk(device -> memory)   // no CPU cycles used
    END WHILE

    // Step 4: DMA signals completion via interrupt (not polling)
    RAISE INTERRUPT(DMA_COMPLETE)
END

ISR DMA_COMPLETE_HANDLER()
    SAVE_CONTEXT()
    status <- DMA.read_status_register()
    IF status == SUCCESS THEN
        mark_buffer_ready(memory_addr, length)
    ELSE
        log_error(status); retry_transfer()
    END IF
    CLEAR_INTERRUPT_FLAG(DMA_COMPLETE)
    RESTORE_CONTEXT()
    RETURN_FROM_INTERRUPT
END ISR
```

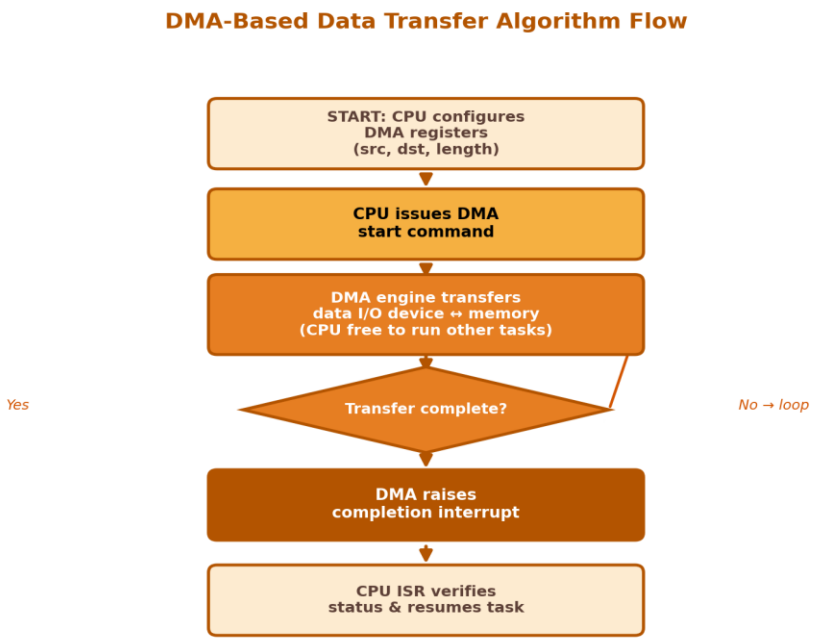

Figure 1: Control flow of the DMA-based transfer algorithm

Explanation: The CPU only performs the one-time setup (source, destination, length) and then issues a single START command. The DMA engine independently drives the device-to-memory transfer in the background, so the CPU is free to execute other instructions for the entire duration of the transfer. The DMA raises exactly one interrupt at completion, which is far cheaper than the per-byte or per-poll CPU cost of Programmed I/O.

Question 2 — Vectored Interrupt Priority Dispatch System

Task: Develop pseudocode for an interrupt system that prioritizes vectored interrupts from multiple devices and dispatches the correct service routine.

Pseudocode:

```
// Priority table: lower number = higher priority
PRIORITY_TABLE[DEVICE_COUNT] <- {device_id, priority_level, vector_addr}

BEGIN INTERRUPT_CONTROLLER_LOOP()
    WHILE TRUE DO
        pending <- READ_IRQ_BITMAP()          // bit set = device requesting service

        IF pending == 0 THEN
            CONTINUE                           // no interrupts pending, idle
        END IF

        // Step 1: pick the pending interrupt with the highest priority
        selected <- NULL
        FOR EACH device IN PRIORITY_TABLE (sorted by priority_level) DO
            IF pending.bit[device.id] == 1 THEN
                selected <- device
                BREAK                          // highest-priority match found
            END IF
```

```
        END FOR

        // Step 2: vectored dispatch - no software search for the ISR
        isr_addr <- VECTOR_TABLE[selected.vector_addr]
        SAVE_CONTEXT()
        JUMP_TO(isr_addr)                    // direct hardware-assisted jump
        // --- selected device's ISR executes here ---
        RESTORE_CONTEXT()
        CLEAR_IRQ_BIT(selected.id)
    END WHILE
END
```

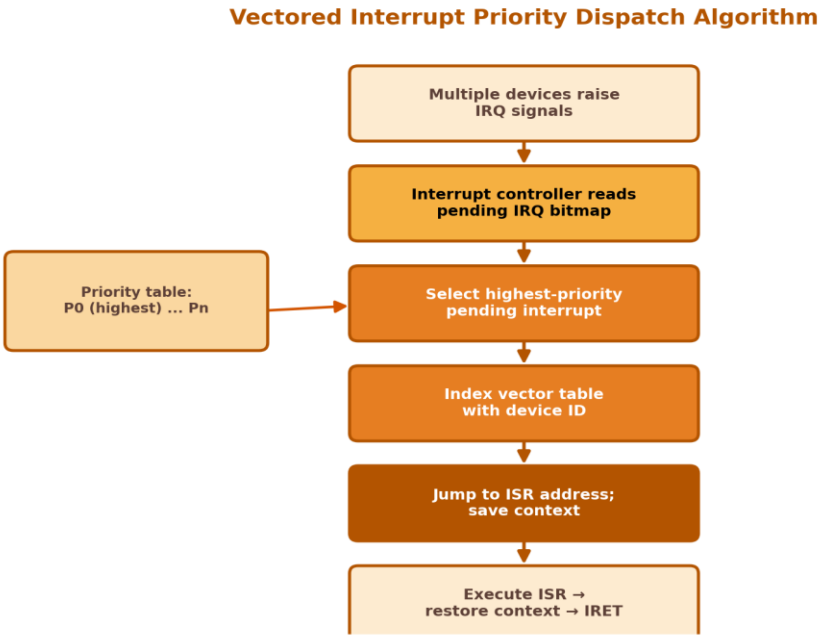

Figure 2: Vectored, priority-ordered interrupt dispatch pipeline

Explanation: Every device is pre-registered in a priority table with its own vector address. When multiple IRQs are pending simultaneously, the controller always services the highest-priority match first. Because dispatch uses a vector table lookup rather than a sequential software poll of each device, the time between interrupt assertion and ISR execution stays short and predictable.

Question 3 — Dynamic Memory-Mapped vs I/O-Mapped Selection

Task: Write pseudocode that dynamically selects between memory-mapped I/O and I/O-mapped I/O based on latency and bandwidth requirements.

Pseudocode:

```
CONST LATENCY_THRESHOLD  <- 5   // microseconds
CONST BANDWIDTH_THRESHOLD <- 100 // MB/s

BEGIN SELECT_IO_MODE(device)
    latency   <- device.measured_latency()
```

```
    bandwidth <- device.measured_bandwidth()

    IF latency < LATENCY_THRESHOLD OR bandwidth > BANDWIDTH_THRESHOLD THEN
        // Fast, bus-pipelined load/store access needed
        device.io_mode <- MEMORY_MAPPED
        MAP_TO_ADDRESS_SPACE(device.registers, MEMORY_REGION)
    ELSE
        // Low-traffic device: isolated I/O space is sufficient
        device.io_mode <- IO_MAPPED
        MAP_TO_ADDRESS_SPACE(device.registers, IO_PORT_SPACE)
    END IF

    BIND_ADDRESS_DECODER(device, device.io_mode)
    RETURN device.io_mode
END

BEGIN PERIODIC_REEVALUATION()             // dynamic switching
    WHILE TRUE DO
        FOR EACH device IN ACTIVE_DEVICES DO
            new_mode <- SELECT_IO_MODE(device)
            IF new_mode != device.io_mode THEN
                RECONFIGURE(device, new_mode)   // switch mapping at runtime
            END IF
        END FOR
        SLEEP(REEVALUATION_INTERVAL)
    END WHILE
END
```

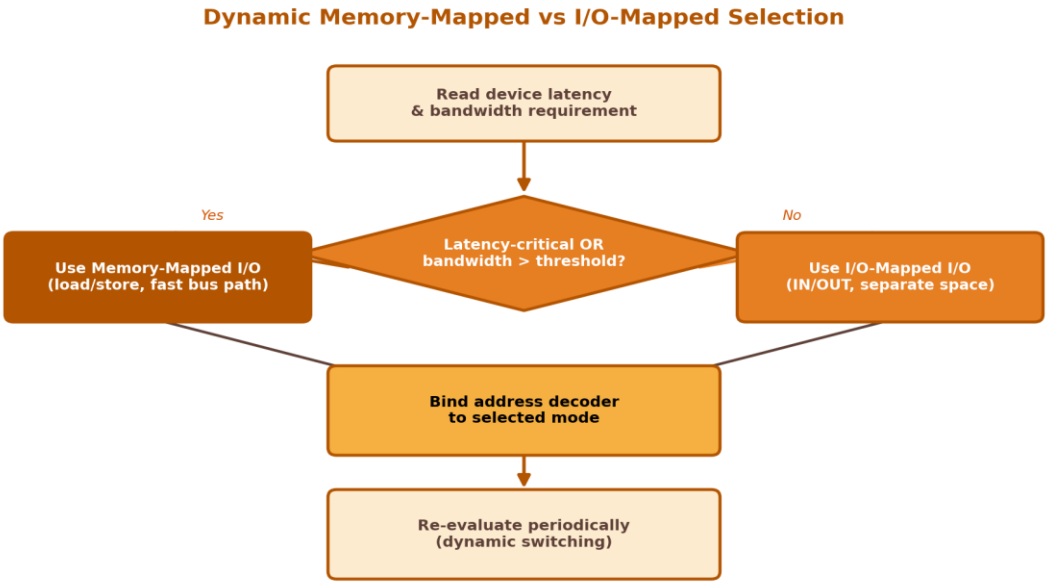

Figure 3: Decision logic for dynamic I/O-mode selection

Explanation: The algorithm continuously measures each device's latency and bandwidth demand. Devices that are latency-critical or bandwidth-heavy are mapped into the memory address space so the CPU can use fast

load/store instructions; the rest use the separate I/O-mapped space. A periodic re-evaluation loop allows the mode to switch at runtime if a device's traffic pattern changes.

Question 4 — Bus Arbitration Algorithm (PCIe, USB, Thunderbolt)

Task: Design pseudocode for a bus arbitration algorithm that fairly manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

Pseudocode:

```
CONST AGING_LIMIT <- 3   // max cycles a request may be deferred
PRIORITY_WEIGHT <- { THUNDERBOLT: 3, PCIE: 2, USB: 1 }

BEGIN BUS_ARBITER()
    round_robin_ptr <- 0
    WHILE TRUE DO
        requests <- COLLECT_PENDING_REQUESTS()   // from all interfaces
        IF requests IS EMPTY THEN CONTINUE

        FOR EACH r IN requests DO
            r.effective_priority <- PRIORITY_WEIGHT[r.interface_type] + r.wait_cycles
        END FOR

        // Anti-starvation: force-boost anything waiting too long
        FOR EACH r IN requests DO
            IF r.wait_cycles >= AGING_LIMIT THEN
                r.effective_priority <- MAX_PRIORITY
            END IF
        END FOR

        winner <- SELECT_HIGHEST(requests, effective_priority,
tiebreak=round_robin_ptr)
        GRANT_BUS(winner.device)
        winner.device.transfer_data()
        RELEASE_BUS(winner.device)

        FOR EACH r IN requests, r != winner DO
            r.wait_cycles <- r.wait_cycles + 1      // age everyone else
        END FOR
        round_robin_ptr <- (round_robin_ptr + 1) MOD DEVICE_COUNT
    END WHILE
END
```

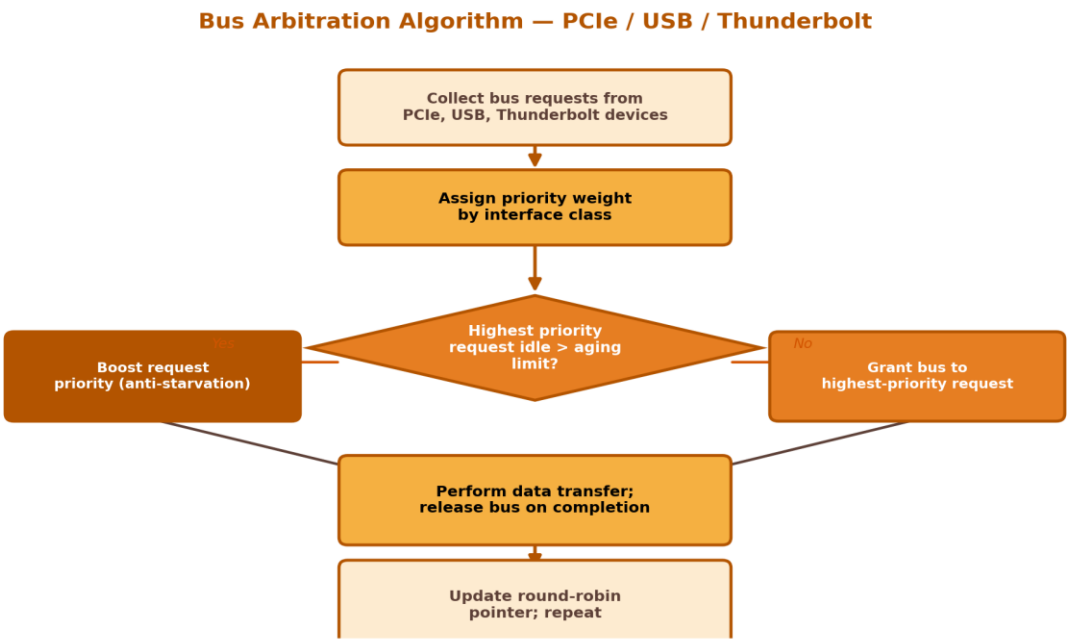

Figure 4: Priority-plus-aging bus arbitration cycle

Explanation: Each interface class starts with a base priority weight (Thunderbolt highest, then PCIe, then USB), but every waiting request also accumulates 'wait_cycles.' Once a request has been deferred past the aging limit, it is force-boosted to the maximum priority so it cannot be starved indefinitely. A round-robin pointer breaks ties fairly among equal-priority requests.

Question 5 — Hybrid I/O: Polling for Low-Speed, DMA+Interrupts for High-Speed

Task: Implement pseudocode for a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Pseudocode:

```
CONST SPEED_THRESHOLD <- 10   // MB/s — boundary between low and high speed

BEGIN HYBRID_IO_SCHEDULER(device_list)
    FOR EACH device IN device_list DO
        IF device.speed > SPEED_THRESHOLD THEN
            REGISTER_HIGH_SPEED(device)
        ELSE
            REGISTER_LOW_SPEED(device)
        END IF
    END FOR

    START_THREAD(HIGH_SPEED_HANDLER)
    START_THREAD(LOW_SPEED_POLLER)
END

BEGIN HIGH_SPEED_HANDLER()                  // DMA + interrupt path
    FOR EACH device IN HIGH_SPEED_DEVICES DO
        CONFIGURE_DMA(device.buffer, device.length)
```

```
        DMA.START(device)
        ENABLE_INTERRUPT(device.completion_line)
    END FOR
END

ISR HIGH_SPEED_COMPLETE(device)
    PROCESS_DATA(device.buffer)
    DMA.START(device)                       // re-arm for next block
END ISR

BEGIN LOW_SPEED_POLLER()                    // polling path
    WHILE TRUE DO
        FOR EACH device IN LOW_SPEED_DEVICES DO
            IF device.status_register == DATA_READY THEN
                PROCESS_DATA(device.read_data())
            END IF
        END FOR
        SLEEP(POLL_INTERVAL)                // avoid busy-wait CPU burn
    END WHILE
END
```

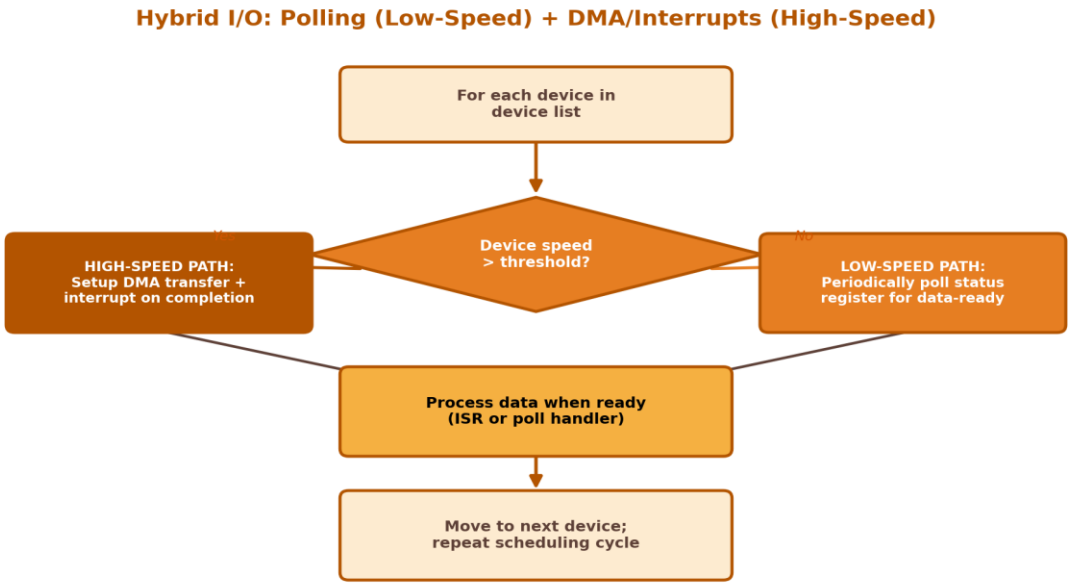

Figure 5: Hybrid scheduler routing devices to polling vs DMA+interrupt paths

Explanation: Devices are classified once by their throughput against a speed threshold. High-speed devices get the full DMA-plus-interrupt treatment so the CPU is not tied up in their bulk transfers. Low-speed devices, where interrupt overhead would outweigh the benefit, are serviced by a lightweight periodic poller with a sleep interval so it does not busy-wait and burn CPU cycles needlessly.

SUMMARY TABLE

S.NO	Algorithm Goal	Core Mechanism Used
1	Move data device → memory with minimal CPU load	DMA setup + single completion interrupt
2	Dispatch correct ISR among competing devices	Priority table + vectored jump to ISR
3	Pick fastest-access I/O technique per device	Threshold check on latency/bandwidth, re-evaluated periodically
4	Fairly share PCIe/USB/Thunderbolt bus	Weighted priority + wait-cycle aging + round robin
5	Serve mixed-speed devices efficiently	Speed threshold routes to polling or DMA+interrupt path